NKT
Tag 4: Generics, Vererbung, Binäre Suchbäume

Sven Hoops, Jonas Goudarzi

DATUM

Gliederung

1 Bäume

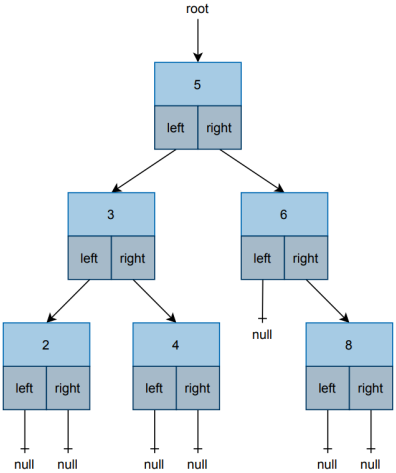
Bäume

Baum:

Bäume

Baum:

- Jedes Element (Knoten): linker und rechter Nachfolger (Kind) (= „Binärbaum“)



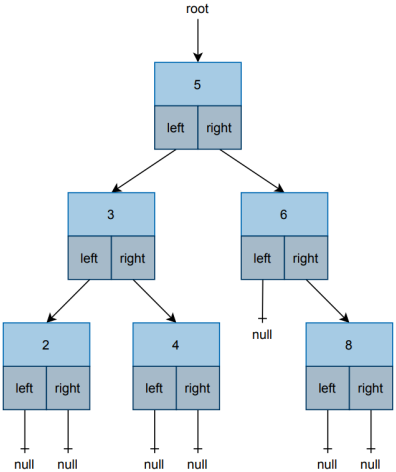
Bäume

Heinrich Heine Universität Düsseldorf

- Ordnung:
  - Alle Knoten im linken Teilbaum kleiner als Knotenwert
  - alle Knoten im rechten Teilbaum größer

  (= „binärer Suchbaum“)
- keine doppelten Werte

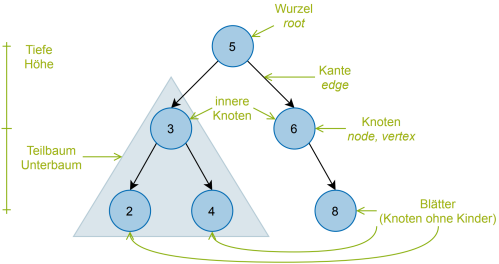
Baum:



Bäume

Baum:

- 2 ist linkes Kind von 3.
- 3 ist Elternknoten von 2 und 4.
- 3 ist die Wurzel des linken Teilbaums von 5.
- Der Baum hat die Tiefe 3.



Bäume - Umsetzung

Heinrich Heine Universität Düsseldorf

```
class BinarySearchTree {

    private class BinaryNode {
    private int element;
    private BinaryNode left, right;


    private BinaryNode(int element) {
        this.element = element;
    }
}

private BinaryNode root;
```

Bäume - Einfügen

Heinrich Heine Universität Düsseldorf

```
1  void insert(int newNumber) {
2      // Sonderfall: leerer Baum
3      if (root == null) {
4          root = new BinaryNode(newNumber);
5          return;
6      }
7
8      BinaryNode parent = null;
9      BinaryNode child = root;
10
11     while (child != null) {
12         parent = child;
13         if (newNumber == child.element) {
14             // Zahl bereits im Baum vorhanden
15             return;
16         } else if (newNumber < child.element) {
17             child = child.left;
18         } else {
19             child = child.right;
20         }
21     }
22     if (newNumber < parent.element) {
23         parent.left = new BinaryNode(newNumber);
24     } else {
25         parent.right = new BinaryNode(newNumber);
26     }
27 }
```

Bäume - Minimum finden

Heinrich Heine Universität Düsseldorf

```
int minimum() {
    if(root == null) {
        throw new java.util.NoSuchElementException();
    }

    BinaryNode current = root;
    while(current.left != null) {
        current = current.left;
    }

    return current.element;
}
```

Einschub: Exceptions

```
throw new [Exception]
```

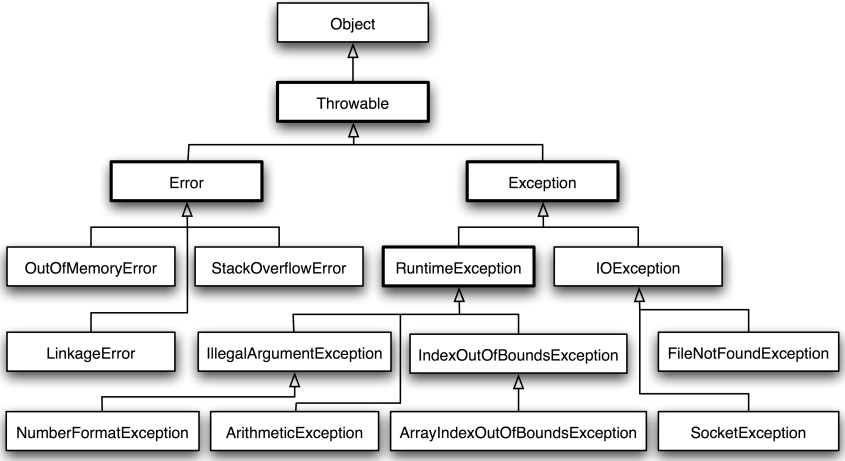
Throwing (werfen) → sorgt für das auslösen einer Exception.

```
Try{
    //Was versucht werden soll
} catch([Exceptiontyp] e) {
    // Code zur Fehlerbehandlung
}
```

- Fängt Exceptions ab
- verhindert Programmabsturz.
- Errors, können auch abgefangen werden! Ist aber fast immer eine schlechte Lösung!

Vorsicht! Alle Exceptions folgen einer Umfangreichen Vererbungshierarchie. Wenn ein zu hoher Obertyp gewählt wird, kann die tatsächliche Exception verdeckt werden oder kann nicht abgefangen werden

Einschub: Exceptions



https://www.protechtraining.com/static/bookshelf/java_fundamentals_tutorial/images/ExceptionClassHierarchy.png